

Block Drill Exam — Set 3 (harder)

Ten blocks where the bug hides behind code that looks careful; several are correct and must be approved

October 2026

How to run it.

Same method. These blocks are harder in one specific way: each contains at least one thing that LOOKS wrong but is fine, and at most two things that are wrong. Your job is to tell them apart with the card, not with instinct. 12 minutes per block. Four of the ten are clean.

Block 1 — Retry with jitter

Requirement: retry `op` up to `n` times with exponential backoff starting at 100 ms, doubling, with $\pm 20\%$ jitter; throw the last error.

```
1  async function retry<T>(op: () => Promise<T>, n: number): Promise<T> {
2    let err: unknown;
3    for (let attempt = 0; attempt < n; attempt++) {
4      try {
5        return await op();
6      } catch (e) {
7        err = e;
8        const base = 100 * 2 ** attempt;
9        const jitter = base * (0.8 + Math.random() * 0.4);
10       if (attempt < n - 1) await new Promise((r) => setTimeout(r, jitter));
11     }
12   }
13   throw err;
14 }
```

Notes

Line	Note
3	<code>attempt</code> 0..n-1: exactly <code>n</code> attempts
5	<code>return await</code> inside <code>try</code> : rejection caught here; correct
8	100, 200, 400...: doubling; correct
9	<code>0.8..1.2</code> $\times$ base: $\pm 20\%$; correct
10	no sleep after the last failure; correct and deliberate
13	<code>err</code> is <code>unknown</code> ; thrown as-is, so the caller gets the original; correct
1	<code>n = 0</code> : loop never runs, throws <code>undefined</code> (edge)

Findings

- Edge only: `n <= 0` throws `undefined` rather than a clear error; validate `n >= 1`.

Confidence 5 · Approve. Everything that looks suspicious (`return await`, no sleep on last, `unknown` rethrow) is correct.

Block 2 — Safe JSON

Requirement: `safeParse(s)` returns `{ ok: true, value }` or `{ ok: false, error: string }`; never throws.

```
1 type Parsed = { ok: true; value: unknown } | { ok: false; error: string };
2 function safeParse(s: string): Parsed {
3   try {
4     return { ok: true, value: JSON.parse(s) };
5   } catch (e) {
6     return { ok: false, error: e.message };
7   }
8 }
```

Notes

Line	Note
1	discriminated union on <code>ok</code> ; good shape
4	<code>JSON.parse</code> is sync, so the <code>try</code> catches it: correct
4	<code>value: unknown</code> , not <code>any</code> : forces callers to narrow; good
6	<code>e</code> is <code>unknown</code> in strict TS; <code>e.message</code> is a compile error; at runtime, <code>JSON.parse</code> always throws a <code>SyntaxError</code> , so <code>.message</code> exists, but a non-Error could in principle be thrown

Findings

- Line 6: `e.message` on `unknown`; narrow: `e instanceof Error ? e.message : String(e)`.

Confidence 4 • Approve with one change.

The behavior is right; the typing is the only hole.

Block 3 — Array chunk

Requirement: split an array into chunks of `size`; last chunk may be shorter; `size` ≥ 1 .

```
1 function chunk<T>(arr: readonly T[], size: number): T[][] {
2   if (!Number.isInteger(size) || size < 1) throw new RangeError("size");
3   const out: T[][] = [];
4   for (let i = 0; i < arr.length; i += size) out.push(arr.slice(i, i + size));
5   return out;
6 }
```

Notes

Line	Note
1	<code>readonly T[]</code> declares the function will not mutate the input
2	validates integer ≥ 1 ; rejects <code>NaN</code> , <code>0</code> , <code>1.5</code>
4	<code>slice</code> copies; <code>i + size</code> past the end is clamped by <code>slice</code> ; last chunk shorter; empty input yields <code>[]</code>

Findings

- None.

Confidence 5 · Approve. The `readonly` is a positive, not a smell.

Block 4 — Memoize by argument

Requirement:

memoize a one-argument function by its argument; cache must not grow beyond 1000 entries.

```
1 function memo<A, R>(fn: (a: A) => R): (a: A) => R {
2   const cache = new Map<A, R>();
3   return (a) => {
4     if (cache.has(a)) return cache.get(a)!;
5     const r = fn(a);
6     cache.set(a, r);
7     if (cache.size > 1000) cache.delete(cache.keys().next().value);
8     return r;
9   };
10 }
```

Notes

Line	Note
2	Map<A, R>: if A is an object, keys compare by identity; two equal-looking objects miss (design note, not a bug for primitives)
4	has then get!: no await between; justified
5	fn(a) sync; if fn throws, nothing is cached: correct
7	evicts the oldest-inserted after exceeding 1000; FIFO, not LRU, but the requirement only says "not grow beyond 1000"; a hit does not refresh position
7	> 1000 after insert means the cache holds 1001 for an instant, then 1000: fine

Findings

- Design note: identity keys for object arguments; FIFO eviction. Neither violates the stated requirement.

Confidence 5 · Approve.

Block 5 — Rate-limited fetcher

Requirement: wrap `fetch` so at most 5 requests are in flight at once; extra calls wait their turn; order of start = order of call.

```
1  const MAX = 5;
2  let active = 0;
3  const waiting: Array<() => void> = [];
4  async function limitedFetch(url: string): Promise<Response> {
5    if (active >= MAX) await new Promise<void>((r) => waiting.push(r));
6    active++;
7    try {
8      return await fetch(url);
9    } finally {
10     active--;
11     waiting.shift()?.();
12   }
13 }
```

Notes

Line	Note
5	if at capacity, park this call by pushing its resolver; <code>await</code> yields
6	increments AFTER the wait: so when a waiter is woken at line 11, <code>active</code> was decremented at 10 and will be incremented at 6: net stays $\leq$ MAX. But: between line 11 waking a waiter and that waiter reaching line 6, a NEW caller can arrive at line 5, see <code>active < MAX</code> (it was just decremented), and go straight through; now two calls proceed on one slot: the limit can be exceeded by the number of concurrent wakeups
8	<code>return await</code> in <code>try</code> so <code>finally</code> runs after the fetch settles: correct
11	<code>shift()?.()</code> wakes the oldest waiter: FIFO order of waiters; correct

Findings

- Lines 5–6 and 10–11: the slot is released (10) before the woken waiter reclaims it (6), so a newcomer can take it; the concurrency limit is soft. Fix: do not decrement when handing the slot directly to a waiter (`if (waiting.length) waiting.shift(!()); else active--;`), or increment `active` on behalf of the waiter before resolving it.

Confidence 3 • Request changes.

Subtle; the structure is right and most reviewers would approve it; the race is real under load.

Block 6 — Discriminated handler

Requirement: handle { type: "create" | "delete"; id: string } events; unknown types must be a compile-time error, not a runtime surprise.

```
1  type Ev = { type: "create"; id: string } | { type: "delete"; id: string };
2  function handle(ev: Ev): string {
3    switch (ev.type) {
4      case "create": return `created ${ev.id}`;
5      case "delete": return `deleted ${ev.id}`;
6      default: {
7        const never: never = ev;
8        return never;
9      }
10   }
11 }
```

Notes

Line	Note
3–5	exhaustive over the union
7	assigning <code>ev</code> to <code>never</code> in <code>default</code> : compiles only if every member of the union is handled above; adding a third <code>type</code> makes this line a compile error: that is the requirement
8	returning <code>never</code> satisfies <code>string</code>

Findings

- None. The `never` trick is the standard exhaustiveness check.

Confidence 5 · Approve.

Block 7 — Transfer between accounts

Requirement: move `amount` from `from` to `to` atomically; reject if insufficient; never leave money created or destroyed.

```
1  async function transfer(from: string, to: string, amount: number) {
2    const a = await db.get(from);
3    if (a.balance < amount) throw new Error("insufficient");
4    await db.set(from, { ...a, balance: a.balance - amount });
5    const b = await db.get(to);
6    await db.set(to, { ...b, balance: b.balance + amount });
7  }
```

Notes

Line	Note
2	awaited; <code>a</code> is a record (or <code>undefined/null</code> if <code>from</code> is unknown: <code>a.balance</code> crashes)
3	boundary correct; <code>amount</code> not validated (negative moves money the other way; <code>NaN</code> passes)
4	debit written; yield point
5	if line 5 or 6 throws (network, unknown <code>to</code>), the debit has already been committed: money destroyed
2–4	two concurrent transfers from the same account both read the same balance and both pass line 3: overdraft
4, 6	spread copies: no mutation of <code>a/b</code> (fine)

Findings

- Lines 4–6: not atomic; failure after 4 loses money; needs a DB transaction (or a compensating credit on failure, which is weaker).
- Lines 2–3: read-await-check-write race; concurrent debits overdraft; a transaction with a row lock, or a conditional update (`WHERE balance >= $amount`).
- Line 3: `amount <= 0` and `NaN` not rejected; negative reverses the transfer.
- Lines 2, 5: unknown ids give `undefined.balance`.

Confidence 1 • Reject.

The one word in the requirement that matters is “atomically”.

Block 8 — Deep freeze

Requirement: `deepFreeze(o)` freezes an object and every nested object; returns the same object; handles cycles.

```
1 function deepFreeze<T extends object>(o: T, seen = new WeakSet<object>()): T {
2   if (seen.has(o)) return o;
3   seen.add(o);
4   for (const v of Object.values(o)) {
5     if (typeof v === "object" && v !== null) deepFreeze(v, seen);
6   }
7   return Object.freeze(o);
8 }
```

Notes

Line	Note
1	<code>WeakSet</code> default parameter: evaluated fresh per top-level call (TS/JS defaults are per call); weakly held so no leak
2–3	cycle guard
5	<code>typeof v === "object" && v !== null</code> : the correct object test (functions are skipped, which is a reasonable choice)
7	freezes after children; <code>Object.freeze</code> returns the same object
4	<code>Object.values</code> skips non-enumerable and symbol keys (edge; acceptable)

Findings

- Edge: arrays are objects, so they are frozen too (correct); functions are not frozen (acceptable; say so); non-enumerable properties are skipped (edge).

Confidence 5 · Approve. The default-parameter `WeakSet` is correct, not the Python shared-default trap.

Block 9 — Timeout wrapper

Requirement: `withTimeout(p, ms)` resolves with `p`'s value or rejects with `TimeoutError` after `ms`; must not leave a pending timer after `p` settles.

```
1 function withTimeout<T>(p: Promise<T>, ms: number): Promise<T> {
2   let timer: ReturnType<typeof setTimeout>;
3   const t = new Promise<never>((_, reject) => {
4     timer = setTimeout(() => reject(new TimeoutError(ms)), ms);
5   });
6   return Promise.race([p, t]).finally(() => clearTimeout(timer));
7 }
```

Notes

Line	Note
3–5	timeout Promise rejects after <code>ms</code> ; <code>timer</code> assigned synchronously inside the executor
6	<code>race</code> : first to settle wins; <code>finally</code> clears the timer either way: clause "no pending timer" satisfied
2	<code>let timer</code> without initializer: TS may complain "used before assigned" at line 6; at runtime it IS assigned because the executor runs synchronously; a <code>!</code> or an initializer fixes the type complaint
6	if <code>p</code> rejects first, the <code>finally</code> still runs and the rejection propagates: correct

Findings

- Nit: line 2 definite-assignment complaint; initialize or `timer!`.

Confidence 5 · Approve.

Block 10 — Event replay

Requirement: `replay(events, state)` applies events in order and returns the new state; the input state must not be mutated; each event is `{ kind: "add" | "remove"; id: string }`.

```
1 function replay(events: Ev[], state: Set<string>): Set<string> {
2   const next = new Set(state);
3   for (const e of events) {
4     if (e.kind === "add") next.add(e.id);
5     else if (e.kind === "remove") next.delete(e.id);
6   }
7   return next;
8 }
```

Notes

Line	Note
2	<code>new Set(state)</code> copies: input not mutated
4–5	both kinds handled; no <code>default</code> , so a third kind would be silently ignored (not a compile error; the requirement does not ask for exhaustiveness here)
7	returns the copy

Findings

- None against the stated requirement. (If exhaustiveness were required, add a `never` default as in Block 6.)

Confidence 5 · Approve.

Scoring yourself

Block	Verdict	What looks wrong but is fine
1	approve	<code>return await</code> ; no sleep on last attempt
2	approve with one typing fix	nothing else
3	approve	<code>readonly</code> parameter
4	approve	FIFO eviction; <code>!</code> after <code>has</code>
5	request changes	the soft limit between release and reclaim
6	approve	assigning to <code>never</code>
7	reject	spreads are fine; atomicity is not
8	approve	<code>WeakSet</code> default parameter
9	approve	<code>let timer</code> before assignment
10	approve	missing <code>default</code>

Six approvals out of ten. If you requested changes on more than four, you are inventing findings; for each one, name the clause it would violate and notice it does not.